



Lab 7: SLAM Visualisation

MCHA4400

Semester 2 2024

Introduction

In this lab, you will develop a visualisation of the marginal distributions of camera and landmark positions that will serve as a foundation for the landmark SLAM rendering aspects of Assignment 1.

GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

Before beginning this lab, merge your earlier work into the Lab 7 template as follows:

- Merge `src/GaussianBase.hpp` from Lab 5 or Lab 6.
- Merge `src/GaussianInfo.hpp` from Lab 6.
- Merge the following in `src/Camera.cpp` from Lab 3:
 - `Camera::calibrate`
 - `Camera::vectorToPixel`
 - `Camera::worldToVector`
 - `Camera::pixelToVector`
 - `Camera::isVectorWithinFOV`
 - `Camera::calcFieldOfView`
 - `ChessboardData::ChessboardData`
 - `ChessboardImage::ChessboardImage`
 - `ChessboardImage::drawBox`

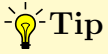
Note

The `Pose` class has been re-implemented in `src/Pose.hpp` as a template class using Eigen (instead of OpenCV) with the scalar type templated to enable automatic differentiation through kinematic expressions. If you previously used `Pose` in your Lab 3 solution, some instances of `Pose` may need to be replaced with `Pose<double>`.

Be mindful that some other changes have been made to these classes, so double check before overwriting any code (use your favourite merge tool).

1. Rotations

In this task you will create templated rotation functions, within `src/rotation.hpp`, to transform between a rotation matrix and its RPY Euler angle parameterisation.

**Tip**

Refer to the Week 3 slides on parameterisations of $\text{SO}(3)$.

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/rotation.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.
- Complete the templated function `rotx`, which returns $\mathbf{R}_x(\phi)$.
- Complete the templated function `roty`, which returns $\mathbf{R}_y(\theta)$.
- Complete the templated function `rotz`, which returns $\mathbf{R}_z(\psi)$.
- Complete the templated function `rp2rot`, which returns $\mathbf{R}(\boldsymbol{\Theta}) = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$ from the vector of Euler angles $\boldsymbol{\Theta} = [\phi \ \theta \ \psi]^\top$.
- Complete the templated function `rot2rp2`, which returns a vector of Euler angles $\boldsymbol{\Theta} = [\phi \ \theta \ \psi]^\top$ for the given $\mathbf{R} \in \text{SO}(3)$.
- Build the application and ensure the unit tests in `test/src/rotation.cpp` pass.

2. Landmarks to features

The world-to-pixel model we developed in Lab 3 maps the location of the world point $\mathbf{r}_{P/N}^n$ to its location in the image $\mathbf{r}_{Q/O}^i$. This was previously implemented with the help of `cv::projectPoints`; however, now we will need to compute the derivatives of this function with respect to the state variables in a SLAM solution.

We will consider the same rational radial distortion, decentering distortion and thin prism distortion models supported by OpenCV, and let $\boldsymbol{\theta}_{\text{dist}} \in \mathbb{R}^m$ be the distortion coefficients of the planar camera model,

$$\begin{bmatrix} u' \\ v' \end{bmatrix} = \underbrace{c \begin{bmatrix} u \\ v \end{bmatrix}}_{\text{radial distortion}} + \underbrace{\begin{bmatrix} 2p_1uv + p_2(r^2 + 2u^2) \\ p_1(r^2 + 2v^2) + 2p_2uv \end{bmatrix}}_{\text{decentering distortion}} + \underbrace{\begin{bmatrix} s_1r^2 + s_2r^4 \\ s_3r^2 + s_4r^4 \end{bmatrix}}_{\text{thin prism distortion}}, \quad (1)$$

where

$$r^2 = u^2 + v^2, \quad u = \frac{x}{z}, \quad v = \frac{y}{z}, \quad \mathbf{r}_{P/C}^c = \begin{bmatrix} x \\ y \\ z \end{bmatrix}. \quad (2)$$

The radial distortion coefficient is given by the following rational model:

$$c = \frac{1 + \alpha}{1 + \beta}, \quad (3)$$

where

$$\alpha = k_1 r^2 + k_2 r^4 + k_3 r^6, \quad (4)$$

$$\beta = k_4 r^2 + k_5 r^4 + k_6 r^6. \quad (5)$$

Using the above expressions, the mapping from world points with respect to C in camera in the camera basis $\{c\}$ by

$$\mathbf{r}_{Q/O}^i = \mathbf{v2p}(\mathbf{r}_{P/C}^c; \boldsymbol{\theta}) = \begin{bmatrix} f_x u' + c_x \\ f_y v' + c_y \end{bmatrix}, \quad (6)$$

where $\boldsymbol{\theta} = \{\boldsymbol{\theta}_{\text{cam}}, \boldsymbol{\theta}_{\text{dist}}\}$ ¹ are the camera intrinsic parameters and $\boldsymbol{\theta}_{\text{cam}} = \{f_x, f_y, c_x, c_y\}$ are the nontrivial elements of the camera matrix

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (7)$$

The world-to-pixel mapping is defined by composing a kinematic transformation from the world point $\mathbf{r}_{P/N}^n$ to the camera vector $\mathbf{r}_{P/C}^c$ with the vector to pixel mapping, which yields

$$\begin{aligned} \mathbf{r}_{Q/O}^i &= \mathbf{v2p}(\mathbf{r}_{P/C}^c; \boldsymbol{\theta}) \\ &= \mathbf{v2p}((\mathbf{R}_c^n)^\top (\mathbf{r}_{P/N}^n - \mathbf{r}_{C/N}^n); \boldsymbol{\theta}) \\ &= \mathbf{v2p}((\mathbf{R}_b^n)^\top ((\mathbf{R}_c^n)^\top (\mathbf{r}_{P/N}^n - \mathbf{r}_{B/N}^n) - \mathbf{r}_{C/B}^b); \boldsymbol{\theta}) \end{aligned} \quad (8a)$$

$$= \mathbf{w2p}(\mathbf{r}_{P/N}^n; \mathbf{T}_b^n, \boldsymbol{\theta}), \quad (8b)$$

where camera position $\mathbf{r}_{C/N}^n \in \mathbb{R}^3$ and camera orientation $\mathbf{R}_c^n \in \mathbf{SO}(3)$ determine the camera pose $\mathbf{T}_c^n \in \mathbf{SE}(3)$ and body position $\mathbf{r}_{B/N}^n \in \mathbb{R}^3$ and body orientation $\mathbf{R}_b^n \in \mathbf{SO}(3)$ determine the body pose $\mathbf{T}_b^n \in \mathbf{SE}(3)$.

For SLAM using point landmarks, we can consider the following state vector:

$$\mathbf{x} = \begin{bmatrix} \boldsymbol{\nu} \\ \boldsymbol{\eta} \\ \mathbf{r}_{1/N}^n \\ \mathbf{r}_{2/N}^n \\ \vdots \\ \mathbf{r}_{n_L/N}^n \end{bmatrix}, \quad (9)$$

where

$$\boldsymbol{\nu} = \begin{bmatrix} \mathbf{v}_{B/N}^b \\ \boldsymbol{\omega}_{B/N}^b \end{bmatrix}, \quad (10)$$

$$\boldsymbol{\eta} = \begin{bmatrix} \mathbf{r}_{B/N}^n \\ \boldsymbol{\Theta}_b^n \end{bmatrix}, \quad (11)$$

¹See [the source code for cvProjectPoints2Internal](#) for more details.

are the body-fixed velocities and world-fixed pose, respectively, $\mathbf{r}_{j/N}^n$ is the position of the j^{th} landmark and n_L is the number of landmarks in the map. The body orientation is parameterised by RPY Euler angles, $\Theta_b^n = [\phi \ \theta \ \psi]^\top$ such that

$$\mathbf{R}_b^n = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi) = e^{\psi\mathbf{S}(\mathbf{e}_3)}e^{\theta\mathbf{S}(\mathbf{e}_2)}e^{\phi\mathbf{S}(\mathbf{e}_1)}. \quad (12)$$

Let $\mathbf{h}_j(\mathbf{x}) \in \mathbb{R}^2$ denote the pixel coordinates of the feature corresponding to the j^{th} landmark as a function of the state $\mathbf{x}$, i.e.,

$$\mathbf{h}_j(\mathbf{x}) = \text{w2p}(\mathbf{r}_{j/N}^n; \mathbf{T}_b^n, \theta). \quad (13)$$

To propagate the uncertainties in the landmark position and camera pose through to uncertainty in the predicted feature location in an image using an affine transform, we will need to compute the Jacobian of $\mathbf{h}_j(\mathbf{x})$ with respect to $\mathbf{x}$. This has the following block structure:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{x}} = \begin{bmatrix} \mathbf{0}_{2 \times 3} & \mathbf{0}_{2 \times 3} & \frac{\partial \mathbf{h}_j}{\partial \mathbf{r}_{B/N}^n} & \frac{\partial \mathbf{h}_j}{\partial \Theta_b^n} & \mathbf{0}_{2 \times 3} & \cdots & \frac{\partial \mathbf{h}_j}{\partial \mathbf{r}_{j/N}^n} & \cdots & \mathbf{0}_{2 \times 3} \end{bmatrix} \in \mathbb{R}^{2 \times (12 + 3n_L)}. \quad (14)$$

Expressions for the analytical derivatives of the non-zero blocks can be found in Appendix B.

Tasks

- a) Complete the implementation of the `MeasurementSLAMPointBundle::predictFeature` template member function in `src/MeasurementSLAMPointBundle.h` and the member function in `src/MeasurementSLAMPointBundle.cpp`.



Info

`MeasurementSLAMPointBundle::predictFeature` is used within `MeasurementSLAMPointBundle::predictFeatureDensity`, which uses `GaussianInfo::affineTransform` to propagate the state distribution through the world to pixel map.

These two functions implement $\mathbf{h}_j(\mathbf{x})$ as the return value and the latter function also writes the Jacobian to `J`, which can be evaluated by one of the following methods:

i) Analytical derivatives only:

Implement the function

```
Eigen::Vector2d Camera::vectorToPixel(const Eigen::Vector3d & rPc, Eigen::Matrix23d & J) const
```

within `src/Camera.cpp` that returns $\mathbf{r}_{Q/O}^i$ and computes $\mathbf{J} = \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^e}$ using the expressions for the analytical Jacobian given in Appendix A. Call this function within

`MeasurementSLAMPointBundle::predictFeature` to obtain $\frac{\partial \mathbf{h}_j}{\partial \mathbf{r}_{j/N}^n}$ and then assemble the blocks of the Jacobian $\frac{\partial \mathbf{h}_j}{\partial \mathbf{x}}$ using the results given in Appendix B.

ii) Mixed analytical derivatives and automatic differentiation:

Implement the `Camera::vectorToPixel` template member function within `src/Camera.h` that returns $\mathbf{r}_{Q/O}^i$, ensuring that all scalar values involved in the derivative are of type given by the template parameter `Scalar`. Then, compute the Jacobian using automatic differentiation

within `MeasurementSLAMPointBundle::predictFeature` to obtain $\frac{\partial \mathbf{h}_j}{\partial \mathbf{r}_{j/N}^n}$ and then assemble the blocks of the Jacobian $\frac{\partial \mathbf{h}_j}{\partial \mathbf{x}}$ using the results given in Appendix B.

iii) **Automatic differentiation only:**

Implement the `Camera::vectorToPixel` template member function within `src/Camera.h` that returns $\mathbf{r}_{Q/O}^i$, ensuring that all scalar values involved in the derivative are of type given by the template parameter `Scalar`. Then, implement the `MeasurementSLAMPointBundle::predictFeature` template member function in `src/MeasurementSLAMPointBundle.h` that implements $\mathbf{h}_j(\mathbf{x})$, ensuring that all scalar values involved in the derivative are of type given by the template parameter `Scalar`. Finally, in the `MeasurementSLAMPointBundle::predictFeature` member function in `src/MeasurementSLAMPointBundle.cpp`, compute the Jacobian of the template function above using automatic differentiation, directly yielding $\frac{\partial \mathbf{h}_j}{\partial \mathbf{x}}$.



Tip

The forward-mode function `gradient` requires a *callable* type as its first argument, i.e., almost anything that can be called like a function using `()`. This includes functions, functors, lambda expressions, function pointers and member function pointers, but not member functions themselves. To compute forward-mode gradients on a member function, we could wrap it in a lambda expression, or use `std::bind` or use a member function pointer. As an example of the latter, if the class `C` has a template member function called `f<>(...)` that we want to compute the gradient w.r.t. its first argument, then we can do the following:

```
double C::f_grad_fwd(const Eigen::VectorXd & x, Eigen::VectorXd & g) const
{
    Eigen::VectorX<autodiff::dual> xdual = x.cast<autodiff::dual>();
    autodiff::dual fdual;
    g = gradient(&C::f<autodiff::dual>, wrt(xdual), at(this, xdual), fdual);
    return val(fdual);           // cast return value to double
}
```

Note that the keyword `this` appears as the first argument in `at()` because under the hood, the member function call `obj.f(x)` is implemented similarly to a non-member function call `f(obj, x)`.



Tip

The reverse-mode function `gradient` requires an `autodiff::var` variable that contains the expression tree built from a forward pass through the function. For example, if the class `C` has a template member function called `f<>(...)` that we want to compute the gradient w.r.t. its first argument, then we can do the following:

```
double C::f_grad_rev(const Eigen::VectorXd & x, Eigen::VectorXd & g) const
{
    Eigen::VectorX<autodiff::var> xvar = x.cast<autodiff::var>();
    autodiff::var fvar = f(xvar);    // Build expression tree
    g = gradient(fvar, xvar);        // Evaluate derivatives from tree
}
```

```
    return val(fvar);           // cast return value to double
}
```

**Tip**

To compute a Jacobian matrix using forward-mode autodifferentiation, see the [jacobian](#) function.

**Tip**

To compute a Jacobian matrix using reverse-mode autodifferentiation, we can compute each row of the Jacobian using the [gradient](#) function as follows, since autodiff doesn't provide a convenient reverse-mode Jacobian:

```
Eigen::VectorXd C::f_jac_rev(const Eigen::VectorXd & x, Eigen::MatrixXd & J) const
{
    Eigen::VectorX<autodiff::var> xvar = x.cast<autodiff::var>();
    Eigen::VectorX<autodiff::var> fvar = f(xvar); // Build expression tree
    J.resize(fvar.size(), xvar.size());
    for (Eigen::Index i = 0; i < fvar.size(); ++i)
        J.row(i) = gradient(fvar(i), xvar); // Evaluate derivatives from tree
    return fvar.cast<double>();           // cast return value to double
}
```

- b) Run `ninja` and ensure the code builds (especially for those using automatic differentiation). Note that there are no unit tests to validate your solution.
- c) (Optional) If you have implemented any of the analytical Jacobians and wish to check your work, consider writing unit tests that use automatic differentiation to generate the oracle data.

3. Confidence regions

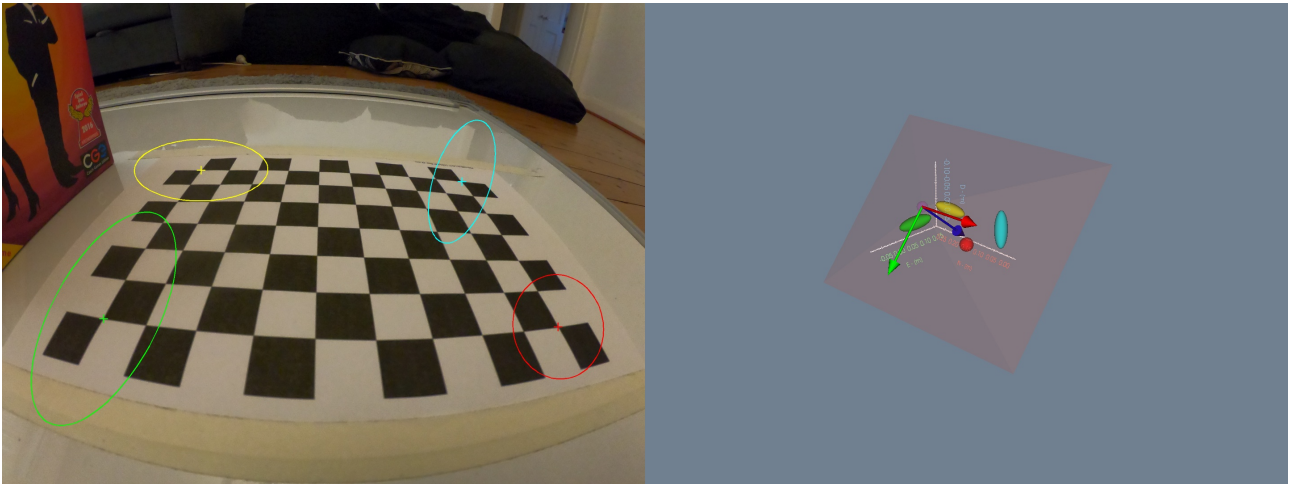


Figure 1: Rendering output for confidence regions of features (left) and camera and landmark positions (right).

Tasks

- Within `src/Plot.cpp`, complete the implementation of the `QuadricPlot::update` function, which calls `GaussianInfo::quadricSurface` to obtain $\mathbf{Q}$ and then populates the coefficients of the `vtkQuadric` instance via `quadric->SetCoefficients(a0, a1, a2, a3, a4, a5, a6, a7, a8, a9)`.

Tip

The `vtkQuadric` class expects a quadric in the following form

$$F(x, y, z) = a_0x^2 + a_1y^2 + a_2z^2 + a_3xy + a_4yz + a_5xz + a_6x + a_7y + a_8z + a_9 = 0,$$

which can be represented in matrix form as

$$F(x, y, z) = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \underbrace{\begin{bmatrix} a_0 & \frac{1}{2}a_3 & \frac{1}{2}a_5 & \frac{1}{2}a_6 \\ \cdot & a_1 & \frac{1}{2}a_4 & \frac{1}{2}a_7 \\ \cdot & \cdot & a_2 & \frac{1}{2}a_8 \\ \cdot & \cdot & \cdot & a_9 \end{bmatrix}}_{\mathbf{Q}} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix},$$

where $\mathbf{Q}$ is a symmetric matrix.

- Copy `config.xml`, `camera.xml` and all the image files within `lab3/data/images` to `lab7/data`.
- Run the application and ensure the confidence ellipses and quadric surfaces match those in Figure 1.

Terminal

```
nerd@basement:~/MCHA4400/lab7/build$ ninja && ./lab7 ../data/config.xml
[Interactive VTK window is displayed]
```

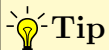
The 3D view can be manipulated by dragging the mouse or using the mouse scroll. Try also holding down **Ctrl** or **Shift** while dragging the mouse. The mouse interactions are documented in the detailed description section of the [vtkInteractorStyleTrackballCamera](#) class and the keyboard commands are documented in the detailed description section of the [vtkInteractorStyle](#) class.

- d) Export all of the calibration images without blocking on the interactor as follows:

Terminal

```
nerd@basement:~/MCHA4400/lab7/build$ ninja && ./lab7 ../data/config.xml -e -i=0
```

Inspect the files written to the `out` directory.



Tip

You may need to modify the plotting code for the Assignment(s). Make use of the [VTK C++ examples](#) to familiarise yourself with the approach for plotting with VTK. You should familiarise yourself with the following class definitions and documentation to strengthen your rendering knowledge.

- [vtkActor](#) — represents an object rendered in the scene, including its properties (colour, shading type, etc.) and position in the world's coordinate system.
- [vtkCamera](#) — defines the view position, focal point and other viewing properties of the scene.
- [vtkImageMapper](#) — provides 2D image display support for VTK.
- [vtkInteractorStyle](#) — the controller for window interactivity.
- [vtkMapper](#) — the geometric representation for an actor. More than one actor may refer to the same mapper.
- [vtkProperty](#) — defines the appearance properties of an actor including colour, transparency, and lighting properties such as specular and diffuse. Also representational properties like wireframe and solid surface.
- [vtkRenderer](#) — coordinates the rendering process involving lights, cameras, and actors.
- [vtkRenderWindow](#) — manages a window on the display device; one or more renderers draw into an instance of `vtkRenderWindow`.

A. Jacobian of vectorToPixel

The gradients of u and v with respect to the input $\mathbf{r}_{P/C}^c$ are given by

$$\frac{\partial u}{\partial \mathbf{r}_{P/C}^c} = \begin{bmatrix} \frac{1}{z} & 0 & -\frac{x}{z^2} \end{bmatrix}, \quad (15)$$

$$\frac{\partial v}{\partial \mathbf{r}_{P/C}^c} = \begin{bmatrix} 0 & \frac{1}{z} & -\frac{y}{z^2} \end{bmatrix}. \quad (16)$$

The gradients of the radius with respect to u and v are given by

$$\frac{\partial r}{\partial u} = (u^2 + v^2)^{-\frac{1}{2}} u, \quad (17)$$

$$\frac{\partial r}{\partial v} = (u^2 + v^2)^{-\frac{1}{2}} v. \quad (18)$$

The gradient of the radial distortion model with respect to r is given by

$$\frac{\partial \alpha}{\partial r} = 2k_1 r + 4k_2 r^3 + 6k_3 r^5. \quad (19)$$

The gradient of the rational distortion model with respect to r is given by

$$\frac{\partial \beta}{\partial r} = 2k_4 r + 4k_5 r^3 + 6k_6 r^5. \quad (20)$$

The gradient of the c with respect to r is given by

$$\frac{\partial c}{\partial r} = \frac{\frac{\partial \alpha}{\partial r}(1 + \beta) - (1 + \alpha)\frac{\partial \beta}{\partial r}}{(1 + \beta)^2}. \quad (21)$$

The gradient of u' with respect to u is given by

$$\frac{\partial u'}{\partial u} = \frac{\partial c}{\partial r} \frac{\partial r}{\partial u} u + c + 2p_1 v + p_2 \left(2r \frac{\partial r}{\partial u} + 4u \right) + 2s_1 r \frac{\partial r}{\partial u} + 4s_2 r^3 \frac{\partial r}{\partial u}. \quad (22)$$

The gradient of u' with respect to v is given by

$$\frac{\partial u'}{\partial v} = \frac{\partial c}{\partial r} \frac{\partial r}{\partial v} u + 2p_1 u + p_2 \left(2r \frac{\partial r}{\partial v} \right) + 2s_1 r \frac{\partial r}{\partial v} + 4s_2 r^3 \frac{\partial r}{\partial v}. \quad (23)$$

The gradient of v' with respect to u is given by

$$\frac{\partial v'}{\partial u} = \frac{\partial c}{\partial r} \frac{\partial r}{\partial u} v + 2p_2 v + p_1 \left(2r \frac{\partial r}{\partial u} \right) + 2s_3 r \frac{\partial r}{\partial u} + 4s_4 r^3 \frac{\partial r}{\partial u}. \quad (24)$$

The gradient of v' with respect to v is given by

$$\frac{\partial v'}{\partial v} = \frac{\partial c}{\partial r} \frac{\partial r}{\partial v} v + c + 2p_2 u + p_1 \left(2r \frac{\partial r}{\partial v} + 4v \right) + 2s_3 r \frac{\partial r}{\partial v} + 4s_4 r^3 \frac{\partial r}{\partial v}. \quad (25)$$

The Jacobian of (6) with respect to $\mathbf{r}_{P/C}^c$ is given by

$$\frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} = \begin{bmatrix} f_x \left(\frac{\partial u'}{\partial u} \frac{\partial u}{\partial \mathbf{r}_{P/C}^c} + \frac{\partial u'}{\partial v} \frac{\partial v}{\partial \mathbf{r}_{P/C}^c} \right) \\ f_y \left(\frac{\partial v'}{\partial u} \frac{\partial u}{\partial \mathbf{r}_{P/C}^c} + \frac{\partial v'}{\partial v} \frac{\partial v}{\partial \mathbf{r}_{P/C}^c} \right) \end{bmatrix}. \quad (26)$$

B. Jacobian of worldToPixel

Applying the chain rule to (8a) yields the Jacobians of $\mathbf{w2p}$ with respect to $\mathbf{r}_{P/N}^n$, $\mathbf{r}_{B/N}^n$ and Θ_b^n as follows:

$$\frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/N}^n} = \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} \frac{\partial \mathbf{r}_{P/C}^c}{\partial \mathbf{r}_{P/N}^n} = \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} (\mathbf{R}_c^b)^\top (\mathbf{R}_b^n)^\top, \quad (27a)$$

$$\frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{B/N}^n} = \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} \frac{\partial \mathbf{r}_{P/C}^c}{\partial \mathbf{r}_{B/N}^n} = -\frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} (\mathbf{R}_c^b)^\top (\mathbf{R}_b^n)^\top, \quad (27b)$$

$$\begin{aligned} \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \Theta_b^n} &= \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} \frac{\partial \mathbf{r}_{P/C}^c}{\partial \Theta_b^n} \\ &= \frac{\partial \mathbf{r}_{Q/O}^i}{\partial \mathbf{r}_{P/C}^c} (\mathbf{R}_c^b)^\top \left[\left(\frac{\partial \mathbf{R}_b^n}{\partial \phi} \right)^\top (\mathbf{r}_{P/N}^n - \mathbf{r}_{B/N}^n) \quad \left(\frac{\partial \mathbf{R}_b^n}{\partial \theta} \right)^\top (\mathbf{r}_{P/N}^n - \mathbf{r}_{B/N}^n) \quad \left(\frac{\partial \mathbf{R}_b^n}{\partial \psi} \right)^\top (\mathbf{r}_{P/N}^n - \mathbf{r}_{B/N}^n) \right]. \end{aligned} \quad (27c)$$

Note

When finding the Jacobian of (13) with respect to Euler angles, we must take the Jacobian of the rotation matrix with respect to its parameters. The rotation matrix can be parametrised by Euler angles as follows:

$$\mathbf{R}(\Theta_b^n) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) = e^{\psi \mathbf{S}(\mathbf{e}_3)} e^{\theta \mathbf{S}(\mathbf{e}_2)} e^{\phi \mathbf{S}(\mathbf{e}_1)}.$$

The Jacobians with respect to each Euler angle are given by

$$\begin{aligned} \frac{\partial \mathbf{R}(\Theta_b^n)}{\partial \psi} &= \frac{\partial \mathbf{R}_z(\psi)}{\partial \psi} \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) = \mathbf{S}(\mathbf{e}_3) \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) = \mathbf{R}_z(\psi) \mathbf{S}(\mathbf{e}_3) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi), \\ \frac{\partial \mathbf{R}(\Theta_b^n)}{\partial \theta} &= \mathbf{R}_z(\psi) \frac{\partial \mathbf{R}_y(\theta)}{\partial \theta} \mathbf{R}_x(\phi) = \mathbf{R}_z(\psi) \mathbf{S}(\mathbf{e}_2) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{S}(\mathbf{e}_2) \mathbf{R}_x(\phi), \\ \frac{\partial \mathbf{R}(\Theta_b^n)}{\partial \phi} &= \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \frac{\partial \mathbf{R}_x(\phi)}{\partial \phi} = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{S}(\mathbf{e}_1) \mathbf{R}_x(\phi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) \mathbf{S}(\mathbf{e}_1). \end{aligned}$$